

Evolucin y Desarrollo de Algoritmos de Grafos para Semforos Inteligentes

Edison Puerto

10 de mayo de 2026

1. Introduccin

Este documento presenta la trayectoria tcnica de los cdigos desarrollados para el proyecto de semaforizacin inteligente. Se muestra desde el modelado bsico de grafos hasta la implementacin del algoritmo de Dijkstra con pesos dinmicos.

2. Fase 1: Modelado Bsico de Grafos

Este es el cdigo inicial que establece la estructura fundamental de nodos y aristas.

```
1 #
2 import networkx as nx
3 import matplotlib.pyplot as plt
4
5 #
6 G = nx.Graph()
7
8 #
9 G.add_node(1)
10 G.add_node(2)
11 G.add_node(3)
12
13 #
14 G.add_edge(1, 2)
15 G.add_edge(2, 3)
16
17 #
18 path = nx.shortest_path(G, source=1, target=3)
19
20 print(f"The shortest path between node 1 and node 3 is: {path}")
21
22 #
23 nx.draw(G, with_labels=True)
24 plt.show()
```

Listing 1: Cdigo inicial de grafos

3. Fase 2: Grafos Dirigidos y Ponderados

Evolucin hacia el uso de pesos (weight) para representar densidades de trfico.

```

1 # Crear un nuevo grafo dirigido para demostrar los pesos
2 import networkx as nx
3 import matplotlib.pyplot as plt
4
5 G_weighted = nx.DiGraph()
6
7 # Aadir aristas con pesos
8 G_weighted.add_edge('A', 'B', weight=1)
9 G_weighted.add_edge('A', 'C', weight=4)
10 G_weighted.add_edge('B', 'C', weight=2)
11 G_weighted.add_edge('B', 'D', weight=5)
12 G_weighted.add_edge('C', 'D', weight=1)
13
14 # Calcular el camino ms corto entre 'A' y 'D'
15 shortest_path_weighted = nx.shortest_path(G_weighted, source='A', target='D', weight='weight')
16 shortest_path_length = nx.shortest_path_length(G_weighted, source='A', target='D', weight='weight')
17
18 print(f"El camino ms corto entre 'A' y 'D' (ponderado) es: {shortest_path_weighted}")
19 print(f"La longitud total del camino ponderado es: {shortest_path_length}")
20
21 # Visualizar el grafo
22 pos = nx.spring_layout(G_weighted)
23 nx.draw(G_weighted, pos, with_labels=True, node_color='lightblue', node_size=2000, font_size=10)
24 labels = nx.get_edge_attributes(G_weighted, 'weight')
25 nx.draw_networkx_edge_labels(G_weighted, pos, edge_labels=labels)
26 plt.title("Grafo con Aristas Ponderadas")
27 plt.show()

```

Listing 2: Grafo con aristas ponderadas

4. Fase 3: Implementacin Manual de Dijkstra

Desarrollo de la lgica del algoritmo utilizando colas de prioridad.

```

1 import heapq
2
3 def dijkstra(graph, start_node):
4     distances = {node: float('infinity') for node in graph}
5     predecessors = {node: None for node in graph}
6     distances[start_node] = 0
7     priority_queue = [(0, start_node)]
8
9     while priority_queue:
10         current_distance, current_node = heapq.heappop(priority_queue)
11         if current_distance > distances[current_node]:
12             continue
13         for neighbor, weight in graph[current_node].items():
14             distance = current_distance + weight
15             if distance < distances[neighbor]:
16                 distances[neighbor] = distance
17                 predecessors[neighbor] = current_node
18                 heapq.heappush(priority_queue, (distance, neighbor))

```

```

19     return distances, predecessors
20
21 def reconstruct_path(predecessors, start_node, end_node):
22     path = []
23     current = end_node
24     while current is not None:
25         path.insert(0, current)
26         current = predecessors[current]
27     if path[0] == start_node:
28         return path
29     return []
30
31 dijkstra_graph = {
32     node: {neighbor: data['weight'] for neighbor, data in G_weighted[
33         node].items()}
34     for node in G_weighted.nodes()
35 }
36 start_node_dijkstra = 'A'
37 end_node_dijkstra = 'D'
38 distances, predecessors = dijkstra(dijkstra_graph, start_node_dijkstra)
39 path_dijkstra = reconstruct_path(predecessors, start_node_dijkstra,
40     end_node_dijkstra)
41 print(f"Camino m s corto de {start_node_dijkstra} a {end_node_dijkstra}:
42     {path_dijkstra}")

```

Listing 3: Implementacin manual de Dijkstra

5. Fase 4: Aplicacin - Semforos Inteligentes

Implementacin final aplicada al contexto de Barrancabermeja.

```

1 import networkx as nx
2 import matplotlib.pyplot as plt
3
4 G = nx.DiGraph()
5 intersecciones = [1, 2, 3, 4]
6 G.add_nodes_from(intersecciones)
7
8 G.add_edge(1, 2, weight=5)
9 G.add_edge(2, 3, weight=10) # Calle con mucho trfico
10 G.add_edge(3, 4, weight=2)
11 G.add_edge(4, 1, weight=4)
12 G.add_edge(2, 4, weight=1) # Atajo detectado por sensor
13
14 inicio = 1
15 destino = 3
16 ruta_optima = nx.shortest_path(G, source=inicio, target=destino, weight=
17     'weight')
18 print(f"Ruta ptima calculada para minimizar tiempo: {ruta_optima}")
19
20 pos = nx.spring_layout(G)
21 plt.figure(figsize=(8, 6))
22 nx.draw(G, pos, with_labels=True, node_color='skyblue', node_size=1500,
23     font_size=12, font_weight='bold', arrowsize=20)

```

```

23 labels = nx.get_edge_attributes(G, 'weight')
24 nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
25 plt.title("Simulacin de Red Vial - Semforos Inteligentes")
26 plt.show()

```

Listing 4: Simulacin final de red vial inteligente

6. Bibliografa

Referencias

- [1] CSDN Wenku. (2024). (Anlisis profundo: mtodos de teora de grafos para el anlisis de redes de transporte urbano).